IMPLEMENTACIÓN DEVOPS 2

INTEGRACIÓN CONTINUA, ENTREGA CONTINUA Y PLAN DE PRUEBAS

SIN-Quick

SQL Server · SSIS · Power BI · Jenkins

Dato	Detalle
Estudiante	Ferdinand Méndez
No. de carné	[0000-00-0000]
Curso	Seminario de Trabajo de Graduación
Universidad	Universidad Mariano Gálvez de Guatemala
Empresa objeto de estudio	Óptica Quick, San Marcos, Guatemala
Entregable	DEVOPS 2 — cierre del Sprint 2
Video en YouTube	[pegar enlace del video]

Índice

1. Introducción **3**
1.1 Adaptación de los lineamientos del curso 3
2. La solución que se integra y despliega **4**
3. El pipeline de CI/CD **4**
3.1 Checkout 5
3.2 Build 5
3.3 Test 5
3.4 Deploy Staging 5
3.5 Extracto del Jenkinsfile 6
4. Entornos y configuración **7**
5. Flujo de trabajo con ramas **7**
6. Pruebas dentro del pipeline **7**
6.1 Ejemplo: prueba unitaria de la regla RN-09 8
6.2 Ejemplo: prueba de aceptación de la conciliación 8
7. Evidencias de ejecución **9**
7.1 Ejecución exitosa 9
7.2 Ejecución fallida 10
8. Hallazgos técnicos del sprint **11**
8.1 SQL Server Express no incluye SSIS ni SQL Server Agent 11
8.2 Un campo UNIQUE que admite nulos 11
8.3 Decisiones de diseño de la carga 11
9. Riesgos y pendientes **11**
10. Conclusiones **12**
11. Referencias **12**

1. Introducción

Este documento describe la segunda implementación DevOps del proyecto SIN-Quick: el pipeline de **integración y entrega continua** en Jenkins y el **plan de pruebas** que ese pipeline ejecuta en cada cambio. Corresponde al cierre del Sprint 2, en el que se construyó el primer proceso de la transformación digital de Óptica Quick: la carga de las ventas del punto de venta y el tablero de ventas y rentabilidad.

El proyecto se desarrolla sobre tres herramientas: **SQL Server** para la base de datos, **SQL Server Integration Services (SSIS)** —el ETL de Visual Studio— para la carga, y **Power BI** para los tableros. El pipeline compila, prueba y despliega esas tres piezas.

1.1 Adaptación de los lineamientos del curso

Los lineamientos del curso están pensados para una aplicación con microservicios. Esta solución es de inteligencia de negocios, por lo que algunos puntos se aplican tal cual y otros se adaptan. La tabla deja explícito cada caso.

Lineamiento	Cómo se aplica en SIN-Quick
CI/CD con Jenkins o GitLab CI	Se aplica: **Jenkins** con un agente Windows, porque SSIS y las herramientas de SQL Server solo corren en Windows.
Etapas checkout → build → test → deploy staging	Se aplica: las cuatro etapas, con build en paralelo de la base y del ETL.
Capturas de ejecuciones exitosas y fallidas	Se aplica: la falla se provoca con un error real en una regla de negocio (sección 7).
Pruebas unitarias, integración, sistema, aceptación, seguridad y rendimiento	Se aplica: los seis tipos, más pruebas de humo del despliegue.
Automatización con pytest	Se aplica: 13 pruebas de pytest que ejecutan la carga real contra SQL Server, con reporte HTML.
Automatización de interfaz con Selenium	Se adapta: Power BI exige autenticación de Microsoft y dibuja sus visuales en un lienzo, así que los datos se prueban en las vistas que alimentan el tablero, y la interfaz se prueba manualmente.
Pruebas manuales de UX	Se aplica: cuatro casos sobre el tablero, con el propietario.
Lenguaje de programación libre	Se aplica en las pruebas y la automatización: **Python** y **PowerShell**, ambos de código abierto. La lógica de datos está en T-SQL, propio del motor.
Sistema de base de datos libre	Se adapta: el proyecto usa **SQL Server**, restricción RI-01 del proyecto. Sus ediciones Developer y Express son gratuitas, pero no son software libre. Se documenta como una desviación consciente del lineamiento.

Respecto de DEVOPS 1: los microservicios contenerizados de aquella entrega se sustituyen por la arquitectura actual de base de datos, ETL e informe, que es la que corresponde a la solución que requiere la empresa. Se conservan el repositorio en GitHub, Azure Boards y la disciplina de Scrum.

2. La solución que se integra y despliega

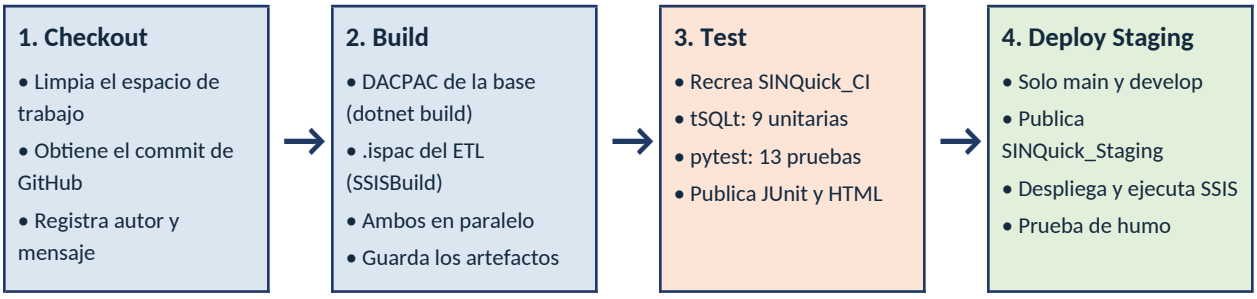
Antes de describir el pipeline conviene ver qué es lo que se compila y se despliega. La base de datos se organiza en cuatro esquemas, cada uno con una responsabilidad:

Esquema	Contenido	Quién lo usa
`stg`	Área de aterrizaje: el archivo del punto de venta tal como llega, todo como texto	El proceso de carga
`etl`	Procedimientos de carga, validación y paso al modelo dimensional	El paquete SSIS y las pruebas
`dbo`	Modelo dimensional: las 17 tablas del diccionario de datos	Nadie consulta directamente
`rpt`	Vistas con los cálculos del negocio: ingreso, margen, alertas, renovación, conciliación	Power BI

La decisión de diseño más importante del sprint fue **dónde vive la lógica**. El paquete SSIS solo orquesta: recorre los archivos de la carpeta de entrada, llama al procedimiento `etl.usp_EjecutarCargaVentas` y mueve cada archivo a *procesados* o a *errores*. Toda la validación y la carga están en T-SQL. Esto permite probar la lógica con tSQLt y pytest sin depender de SSIS, y permite que en una instalación con SQL Server Express —que no incluye SSIS— la misma carga se ejecute con un script de PowerShell.

Todo el esquema está en un **proyecto de base de datos** (`SINQuick.DB.sqlproj`) que compila a un archivo DACPAC. El DACPAC es la definición completa de la base; SqlPackage lo compara contra el destino y aplica solo las diferencias. Así la base de pruebas, la de staging y la de producción quedan siempre iguales a lo que dice el repositorio.

3. El pipeline de CI/CD



Si cualquier etapa falla, el pipeline se detiene en rojo y las siguientes no se ejecutan: un cambio que rompe una prueba nunca llega a staging.

Figura 1. Etapas del pipeline de SIN-Quick en Jenkins

El pipeline se define en el archivo `Jenkinsfile` en la raíz del repositorio. Se configuró como un *Multibranch Pipeline*: Jenkins detecta cada rama y ejecuta el pipeline en cada push. Cada etapa llama a un script de PowerShell de la carpeta `ci/`, de modo que cualquier paso se puede reproducir a mano en el equipo del desarrollador.

3.1 Checkout

Limpia el espacio de trabajo y obtiene el commit que disparó la ejecución. Limpiar primero es deliberado: evita que un artefacto de una ejecución anterior haga pasar una compilación que en realidad está rota.

3.2 Build

Compila las dos piezas en paralelo:

Rama paralela	Herramienta	Artefacto
Base de datos	`dotnet build` sobre el proyecto SDK de SQL (Microsoft.Build.Sql)	`SINQuick.DB.dacpac`
ETL	`SSISBuild.exe` sobre el proyecto de Integration Services	`SINQuick.ETL.ispac`

Compilar la base detecta errores antes de tocar ningún servidor: una vista que referencia una columna inexistente o un procedimiento con un error de sintaxis hacen fallar el build. Los artefactos se archivan en Jenkins con su huella digital, de modo que se sabe exactamente qué versión se desplegó en cada entorno.

3.3 Test

Tiene tres pasos secuenciales:

1. **Desplegar la base de pruebas.** SqlPackage publica el DACPAC en `SINQuick_CI` con la opción de crearla desde cero. Cada ejecución empieza con una base limpia.
2. **Pruebas unitarias con tSQLt.** Instala tSQLt en la base de pruebas, carga las dos clases de prueba y ejecuta las 9 pruebas. Los resultados se exportan en formato JUnit.
3. **Pruebas con pytest.** Ejecuta 13 pruebas que llaman a la carga real con archivos CSV y verifican el resultado en las tablas y vistas. Genera el reporte HTML, un JUnit y un JSON con el resultado de cada caso.

Al terminar, siempre —aun si algo falló— Jenkins publica los resultados JUnit en la vista **Test Result** y el reporte HTML de pytest como un enlace en la ejecución.

3.4 Deploy Staging

Solo se ejecuta para las ramas `develop` y `main`. Las ramas de trabajo (`feature/*`) se compilan y se prueban, pero no se despliegan: así staging siempre refleja código ya integrado.

4. **Publica la base** `SINQuick_Staging` con SqlPackage y la opción `BlockOnPossibleDataLoss`: si un cambio en el esquema fuera a borrar datos, el despliegue se detiene en lugar de perderlos.
5. **Despliega el paquete SSIS** en el catálogo SSISDB, en la carpeta `SINQuick_Staging`.
6. **Ejecuta el paquete una vez** con un archivo de muestra y espera su resultado. Si el paquete no termina con éxito, la etapa falla.
7. **Corre la prueba de humo** (CP-23), que verifica que existan los esquemas, las vistas, los procedimientos, los roles, los parámetros y el calendario.

8. **Publica el informe en Power BI** (opcional). Requiere Power BI Pro y una entidad de servicio; se activa con el parámetro `PUBLICAR_POWERBI`.

3.5 Extracto del Jenkinsfile

```
stage('Test') {
    stages {
        stage('Desplegar base de pruebas') {
            steps { powershell '.\\ci\\publicar-db.ps1 -Base $env:BD_CI -Recrear' }
        }
        stage('Unitarias (tSQLt)') {
            steps { powershell '.\\ci\\pruebas-tsqlt.ps1 -Base $env:BD_CI' }
        }
        stage('Integración, sistema, aceptación, seguridad y rendimiento (pytest)') {
            steps { powershell '.\\ci\\pruebas-pytest.ps1 -Base $env:BD_CI' }
        }
    }
    post {
        always {
            junit testResults: 'reportes/tsqlt.xml, reportes/pytest.xml'
            publishHTML(target: [reportName: 'Reporte de pruebas (pytest)',
                                 reportDir: 'reportes', reportFiles: 'pytest.html'])
        }
    }
}
```

4. Entornos y configuración

Entorno	Base de datos	Edición de SQL Server	Quién lo actualiza	Datos
CI	SINQuick_CI	Developer	El pipeline, en cada ejecución	Sintéticos; se recrea siempre
Staging	SINQuick_Staging	Developer	El pipeline, en develop y main	Catálogo de referencia y muestras
Producción	SINQuick	Por decidir (sección 8.1)	Despliegue aprobado a mano	Reales de la óptica

Secretos. El repositorio no contiene contraseñas. Las bases se acceden con autenticación de Windows, usando la cuenta con que corre Jenkins. Las credenciales de Power BI se guardan en el almacén de credenciales de Jenkins y llegan al script solo durante su ejecución, mediante `withCredentials`.

Parámetros del negocio. El ciclo de renovación, los días sin movimiento y el umbral de rechazo viven en la tabla `ParametroRegla`. El script posterior al despliegue los crea si no existen, pero **nunca sobrescribe** un valor existente: si el propietario ajustó un parámetro, un despliegue no lo devuelve a su valor inicial.

5. Flujo de trabajo con ramas

Rama	Qué contiene	Pipeline
`feature/*`	El trabajo de una historia de usuario	Checkout, Build y Test
`develop`	Historias terminadas e integradas	Las cuatro etapas: se despliega a staging
`main`	Versiones aprobadas en la Sprint Review	Las cuatro etapas: base de la versión de producción

Cada rama `feature/*` se integra a `develop` mediante *pull request*, que solo se acepta con el pipeline en verde. Los mensajes de commit incluyen el identificador de la historia de Azure Boards (`AB#<id>`), lo que vincula cada cambio con su historia de usuario.

6. Pruebas dentro del pipeline

El plan de pruebas completo se entrega como documento aparte. Aquí se resume cómo se integra al pipeline.

Tipo	Herramienta	Casos	Etapa
Unitarias	tSQLt	CP-01 a CP-09	Test
Integración	pytest	CP-10 a CP-12	Test
Sistema	pytest	CP-13 y CP-14	Test
Aceptación	pytest	CP-15 a CP-17	Test
Seguridad	pytest	CP-18 a CP-20	Test

Tipo	Herramienta	Casos	Etapa
Rendimiento	pytest	CP-21 y CP-22	Test
Humo	pytest	CP-23	Deploy Staging
Manuales (UX)	Observación	CP-24 a CP-27	Sprint Review

6.1 Ejemplo: prueba unitaria de la regla RN-09

La prueba aísla las tablas con `FakeTable`, crea cuatro clientes con distinta condición de contacto y compara la vista contra el resultado esperado:

```
CREATE PROCEDURE [pruebasReglas].[test CP-03 RN-09 solo es contactable quien ...]
AS
BEGIN
    EXEC tSQLt.FakeTable 'dbo.Cliente';
    EXEC tSQLt.FakeTable 'dbo.Venta';
    INSERT INTO dbo.Cliente (idCliente, nombreCompleto, telefono, consentimientoContacto, ...)
    VALUES (1, 'Contactable', '50255510001', 1, ...), (2, 'Sin consentimiento', ...,
0, ...), ...;

    SELECT idCliente, contactable, motivoExclusion INTO #actual FROM
rpt.vw_ClientesRenovacion;
    INSERT INTO #esperado VALUES (1, 1, NULL), (2, 0, 'SIN CONSENTIMIENTO'), ...;
    EXEC tSQLt.AssertEqualsTable '#esperado', '#actual';
END;
```

6.2 Ejemplo: prueba de aceptación de la conciliación

pytest genera un archivo como el del punto de venta, ejecuta la carga real y compara el total del mes en la vista contra el total del archivo:

```
@pytest.mark.caso('CP-15')
def test_total_del_mes_coincide_con_el_punto_de_venta(base_limpia, archivo, obtenido):
    cargar(conn, archivo('ventas_mes.csv', filas))
    total_pos = sum(f[6] * f[7] - f[8] for f in filas if f[9] == 0)
    total_dw = uno(conn, 'SELECT totalFacturado FROM rpt.vw_ConciliacionMensual ...')
    assert total_dw == pytest.approx(total_pos)
```

Cada prueba registra su resultado observado mediante el fixture `obtenido`. Ese texto aparece en el reporte y, con el script `completar_resultados.py`, se copia a la columna *Resultado obtenido* del plan de pruebas. Así el plan no se llena a mano ni de memoria.

7. Evidencias de ejecución

Las capturas se obtienen ejecutando el pipeline en el agente de Jenkins, siguiendo la guía `docs/ci/guia-capturas.md`. La ejecución fallida no se simula: se introduce un error real en una regla de negocio y se deja que las pruebas lo detecten.

7.1 Ejecución exitosa

Insertar captura: Stage View con las cuatro etapas en verde

Jenkins → pipeline → rama develop → Stage View

Figura 2. Ejecución exitosa del pipeline

Insertar captura: Test Result con las pruebas tSQLt y pytest aprobadas

Jenkins → ejecución → Test Result

Figura 3. Resultados de las pruebas publicados en Jenkins

Insertar captura: Reporte HTML de pytest

Jenkins → ejecución → Reporte de pruebas (pytest): encabezado de entorno y columnas Caso y Tipo

Figura 4. Reporte HTML de pytest

7.2 Ejecución fallida

En la rama `feature/error-rn09` se eliminó de la vista `rpt.vw_ClientesRenovacion` la condición de consentimiento. Es el peor error posible en este sistema: permitiría contactar a clientes que no dieron permiso. La prueba CP-03 lo detecta, la etapa Test queda en rojo y **la etapa Deploy Staging no se ejecuta**.



Figura 5. Ejecución fallida del pipeline

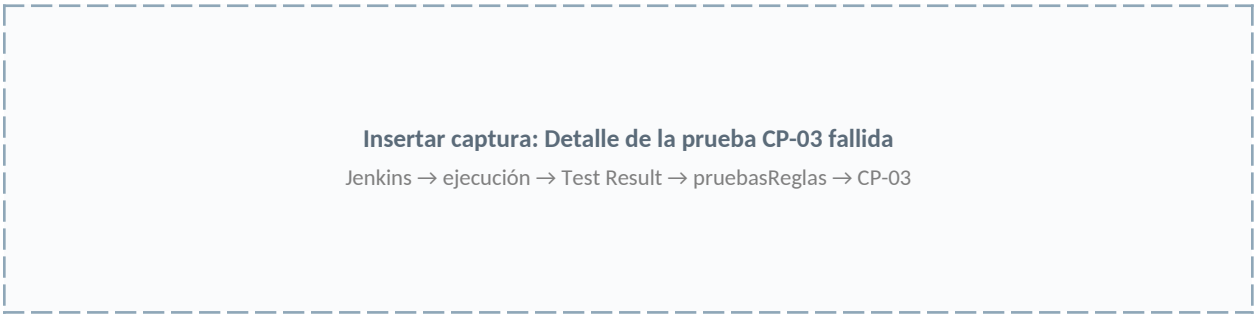


Figura 6. La prueba unitaria que detectó el error

Después de las capturas, el cambio se revierte con `git revert` y el pipeline vuelve a verde, lo que completa la demostración: el error se detectó, se contuvo antes de staging y se corrigió.

8. Hallazgos técnicos del sprint

Construir y automatizar la solución expuso varios supuestos de los documentos anteriores que no se sostenían. Se registran porque cambian decisiones del proyecto.

8.1 SQL Server Express no incluye SSIS ni SQL Server Agent

La arquitectura documentada combinaba SQL Server Express con un paquete SSIS programado en SQL Server Agent. Esa combinación no es posible: ni SSIS ni el Agent vienen en la edición Express. Para CI y staging se usa la edición **Developer**, que es gratuita para desarrollo y pruebas e incluye ambos. Para producción quedan dos opciones, que debe decidir el propietario:

Opción	Costo	Carga nocturna
SQL Server Standard	Licencia de pago	Paquete SSIS programado en SQL Server Agent
SQL Server Express	Gratuito	Script `Ejecutar-CargaVentas.ps1` en el Programador de tareas de Windows, que llama a los mismos procedimientos

Haber puesto la lógica en T-SQL y no en el paquete es lo que hace viable la segunda opción sin reescribir nada.

8.2 Un campo UNIQUE que admite nulos

El diccionario definía el teléfono del cliente como único y a la vez opcional. En SQL Server una restricción UNIQUE solo admite un valor nulo, por lo que el segundo cliente sin teléfono habría hecho fallar la carga. Se resolvió con un índice único filtrado (`WHERE telefono IS NOT NULL`).

8.3 Decisiones de diseño de la carga

- **La carga nace como FALLIDA** en la bitácora y solo cambia a EXITOSA o PARCIAL al terminar. Si el proceso se interrumpe, nunca aparece como exitosa una carga que no terminó.
- **Umbral de rechazo.** Si más del 5 % de las líneas son inválidas, la carga se detiene sin tocar el data warehouse. Un archivo con tantos errores indica un problema de origen, y cargarlo parcialmente mostraría cifras engañosas.
- **Carga idempotente.** Las facturas ya cargadas se omiten sin contarse como error, por lo que volver a procesar un archivo no duplica ventas.
- **Costo congelado.** El costo de cada línea se guarda al momento de la carga, para que el margen histórico no cambie cuando cambie el costo del producto.

9. Riesgos y pendientes

Pendiente	Por qué importa	Cuándo se resuelve
Validar el formato real de la exportación del punto de venta	Las pruebas usan un formato supuesto	Primer día del Sprint 3
Decidir la edición de SQL Server para	Determina cómo corre la carga nocturna	Con el propietario, antes del

Pendiente	Por qué importa	Cuándo se resuelve
producción		despliegue a producción
Licencia de Power BI Pro para publicar	Sin ella la publicación en el servicio no es posible	Antes de habilitar PUBLICAR_POWERBI
Ejecutar el pipeline y registrar los resultados	Las capturas y el resultado obtenido salen de una ejecución real	En el agente de Jenkins, siguiendo docs/ci

10. Conclusiones

El aporte principal de este sprint no es el pipeline en sí, sino lo que el pipeline hace imposible: que un cambio que rompe una regla de negocio llegue a un entorno donde alguien pueda tomar decisiones con él. En una solución de inteligencia de negocios, donde el riesgo es una cifra equivocada que nadie detecta, eso vale más que cualquier funcionalidad nueva.

La automatización también funcionó como revisión técnica: construir de verdad la carga reveló que la combinación de Express con SSIS y Agent no era posible, y que un campo del diccionario violaba una regla del motor. Ninguno de esos problemas se veía en los documentos; aparecieron en cuanto hubo que compilar y desplegar.

Finalmente, la decisión de mantener la lógica en T-SQL y dejar al paquete SSIS solo como orquestador fue la que permitió probar casi todo de forma automática y la que deja abierta una ruta de producción sin costo de licencias.

11. Referencias

- Microsoft. *SQL projects (Microsoft.Build.Sql)*. https://learn.microsoft.com/sql/tools/sql-database-projects/
- Microsoft. *SqlPackage*. https://learn.microsoft.com/sql/tools/sqlpackage/
- Microsoft. *Standalone SQL Server Integration Services DevOps tools*. https://learn.microsoft.com/sql/integration-services/devops/
- Microsoft. *Editions and supported features of SQL Server 2022*. https://learn.microsoft.com/sql/sql-server/editions-and-components-of-sql-server-2022
- tSQLt. *Database unit testing for SQL Server*. https://tsqlt.org
- Jenkins. *Pipeline syntax*. https://www.jenkins.io/doc/book/pipeline/syntax/
- pytest y pytest-html. https://docs.pytest.org · https://pytest-html.readthedocs.io